



**TECNOLÓGICO
NACIONAL DE MÉXICO**



Integrantes:

Herrera Quiñones Abraham Gael

Ochoa Aviles Edgar

Carrera:

Ingeniería En Sistemas Computacionales

Materia:

Tópicos De IA

Tarea:

Sistema de detección de placas vehiculares

Profesor:

Zuriel Dathan Mora Felix

Grupo:

12:00 - 13:00 PM

Fecha:

30/11/2025

GITHUB LINK:

[https://github.com/edgarochoa00/Topicos-De-Inteligencia-Artificial/tree/main/MODULO%](https://github.com/edgarochoa00/Topicos-De-Inteligencia-Artificial/tree/main/MODULO%204)

Índice

1. Resumen
2. Introducción
3. Planteamiento del problema
4. Objetivo general y objetivos específicos
5. Marco teórico
6. Modelo lógico del sistema
7. Propuesta de solución
8. Desarrollo e implementación
9. Datos y cartografía de apoyo (Diseño de BD)
10. Métricas y protocolo experimental
11. Resultados esperados
12. Discusión
13. Cronograma
14. Conclusiones
15. Referencias

1. Resumen

El presente proyecto tiene como propósito fundamental el desarrollo e implementación de un sistema integral de detección y reconocimiento de matrículas vehiculares (LPR - License Plate Recognition). Este sistema está diseñado para identificar de manera automática a los propietarios de vehículos mediante la integración de técnicas avanzadas de visión artificial y gestión robusta de bases de datos. El núcleo del proyecto reside en la automatización del flujo de trabajo que va desde la captura de imágenes estáticas o tramas de video hasta la vinculación de estos datos visuales con la información administrativa registrada de los conductores y sus unidades.

Para lograr este objetivo, el sistema implementa una arquitectura de software híbrida y distribuida. Se utiliza el lenguaje Python como motor de procesamiento de imágenes debido a su amplia gama de librerías científicas, mientras que Node.js gestiona la lógica del servidor y la comunicación asíncrona. En el apartado de Inteligencia Artificial, se emplean algoritmos de Redes Neuronales Convolucionales (CNN) mediante la librería EasyOCR, la cual ha sido seleccionada por su eficacia en entornos no controlados. Este motor de reconocimiento es potenciado por un pipeline de preprocesamiento de imágenes diseñado específicamente con OpenCV, que incluye técnicas de mejora de contraste adaptativo (CLAHE), filtrado bilateral para reducción de ruido y algoritmos de corrección de perspectiva geométrica. Finalmente, la solución integra un módulo de validación lógica que utiliza expresiones regulares para filtrar matrículas mexicanas y algoritmos de lógica difusa para descartar falsos positivos, asegurando una alta fiabilidad en los resultados entregados al usuario final.

(Referencias: EasyOCR Documentation, 2023; OpenCV Team, 2023)

2. Introducción

La identificación automática de vehículos se ha consolidado como un componente crítico e indispensable en la infraestructura de los sistemas modernos de transporte inteligente (ITS) y en las estrategias de seguridad pública a nivel global. El crecimiento exponencial del parque vehicular en las zonas urbanas ha hecho que los métodos tradicionales de control, basados en la inspección visual humana, sean obsoletos, ineficientes y costosos. Tradicionalmente, la verificación de matrículas y la validación de la propiedad del vehículo dependían de la revisión manual por parte de agentes de tránsito o guardias de seguridad, un proceso inherentemente lento, limitado por la fatiga visual y propenso a errores humanos significativos, especialmente en condiciones de alto flujo vehicular o situaciones de estrés.

Este proyecto aborda la problemática mediante la creación de un sistema "End-to-End" (de extremo a extremo) que fusiona la potencia del aprendizaje profundo (Deep Learning) para el análisis semántico de imágenes con la robustez y consistencia de los sistemas de gestión de bases de datos relacionales (RDBMS). A diferencia de los sistemas OCR clásicos que dependen de la coincidencia de plantillas rígidas, este enfoque utiliza redes neuronales capaces de generalizar características, lo que permite al sistema adaptarse a diferentes tipografías y

condiciones ambientales. Utilizando librerías de vanguardia como easyocr para la inferencia neuronal y opencv-python para la manipulación matricial de imágenes, se busca ofrecer una solución tecnológica escalable. El sistema está diseñado para agilizar la identificación vehicular en tiempo real, manteniendo su operatividad incluso bajo condiciones adversas como iluminación variable, sombras proyectadas o ángulos de cámara no óptimos, factores que comúnmente degradan el rendimiento de los sistemas comerciales actuales.

(Referencia: Rosebrock, 2018)

3. Planteamiento del problema

La gestión del control vehicular en accesos controlados, estacionamientos públicos y puntos de seguridad estratégica enfrenta desafíos técnicos y operativos considerables. Los métodos actuales suelen requerir una intervención manual constante, donde el operador debe transcribir la matrícula observada en un sistema informático. Este paso manual no solo introduce un retraso significativo en el proceso (latencia operativa), limitando el volumen de vehículos que pueden ser procesados por hora, sino que también genera cuellos de botella físicos que resultan en congestión vehicular. Además, el error humano en la transcripción de caracteres alfanuméricos —como confundir el número '0' con la letra 'O', o el '8' con la letra 'B'— corrompe la integridad de los registros de seguridad.

Desde una perspectiva de visión por computadora, el problema es aún más complejo debido a la naturaleza no estructurada del entorno. La variabilidad en las condiciones de captura es la norma, no la excepción. Las imágenes obtenidas en escenarios reales suelen presentar desenfoque de movimiento (motion blur) si el vehículo no se detiene completamente, baja resolución si la cámara está distante, o condiciones de iluminación extremas (contraluz al amanecer o ruido digital en tomas nocturnas). Además, las matrículas pueden estar sucias, dañadas, o presentar marcos y calcomanías que confunden a los algoritmos de segmentación tradicionales. El problema técnico central se define, por tanto, como la necesidad de extraer información textual estructurada (la cadena de caracteres de la matrícula) a partir de datos visuales ruidosos y no estructurados (la matriz de píxeles de la imagen), y vincularla de forma precisa e inequívoca con una entidad en la base de datos. Es imperativo minimizar los falsos positivos generados por textos incidentales en el vehículo, como rótulos de marca, publicidad o señalización de emergencia, que a menudo son interpretados erróneamente como matrículas por sistemas ingenuos.

(Referencia: Szeliski, 2010)

4. Objetivo general y objetivos específicos

Objetivo general

Desarrollar un sistema de visión artificial de alto rendimiento, integrado con una base de datos relacional, capaz de detectar, segmentar y reconocer con precisión matrículas vehiculares en imágenes digitales, asociándose automáticamente con sus respectivos registros de propietarios para optimizar la gestión de seguridad y el

control de acceso vehicular.

Objetivos específicos

1. **Diseño de Persistencia de Datos:** Diseñar e implementar un esquema de base de datos normalizado en Microsoft SQL Server que almacena de manera eficiente la información de vehículos, propietarios e historial de detecciones, garantizando la integridad referencial y la rapidez en las consultas mediante índices adecuados.
2. **Preprocesamiento Avanzado:** Construir un módulo de visión artificial en Python capaz de preprocesar las imágenes de entrada mediante técnicas de filtrado morfológico, ecualización de histograma adaptativo (CLAHE) y binarización, con el fin de aislar la región de interés (ROI) de la placa y maximizar el contraste de los caracteres frente al fondo.
3. **Motor de Reconocimiento Inteligente:** Implementar y optimizar un motor OCR basado en redes neuronales (EasyOCR) que incluya validación lógica posterior mediante expresiones regulares (Regex) para patrones de matrículas mexicanas, descartando lecturas erróneas o textos decorativos.
4. **Integración de Sistemas:** Desarrollar una API RESTful o un mecanismo de comunicación entre procesos que conecte el módulo de detección (Python) con el backend del servidor (Node.js), permitiendo la recuperación y visualización de los datos del propietario (nombre, contacto, estado) en tiempo real tras la detección de una placa válida.
(Referencias: Date, 2004; EasyOCR Documentation, 2023)

5. Marco teórico

Para comprender el funcionamiento del sistema propuesto, es necesario profundizar en los fundamentos matemáticos y tecnológicos que lo sustentan. El sistema se apoya en tres pilares principales: procesamiento digital de imágenes, aprendizaje profundo y teoría de bases de datos.

5.1. Preprocesamiento de Imágenes y Visión Computacional

El preprocesamiento es la etapa crítica donde se mejora la calidad de la imagen original antes de ser alimentada a la red neuronal. Una imagen digital se representa matemáticamente como una matriz de intensidades $I(x,y)$.

- **Conversión a Escala de Grises:** El primer paso es reducir la dimensionalidad del espacio de color, transformando la imagen RGB a escala de grises. Esto elimina la información de crominancia que suele ser irrelevante para la detección de caracteres y reduce la carga computacional en un tercio.
- **CLAHE (Contrast Limited Adaptive Histogram Equalization):** A diferencia de la ecualización de histograma global, que puede amplificar el ruido en regiones homogéneas, CLAHE divide la imagen en pequeñas regiones o "tiles". En cada una, se calcula un histograma local y se redistribuyen los valores de intensidad. Esto es crucial para manejar sombras parciales sobre la placa. El límite de contraste (clip limit) evita que el ruido inherente al sensor

de la cámara se amplifique excesivamente.

- **Filtrado Bilateral:** Para la reducción de ruido, se opta por un filtro bilateral en lugar de un desenfoque gaussiano simple. El filtro bilateral suaviza las texturas (promediando píxeles cercanos espacialmente) pero preserva los bordes afilados (evitando promediar píxeles con diferencias de intensidad grandes). Esto es vital para mantener la nitidez de los bordes de los caracteres alfanuméricos.

(Referencia: OpenCV Team, 2023)

5.2. Reconocimiento Óptico de Caracteres (OCR) con Deep Learning

El sistema utiliza EasyOCR, una biblioteca moderna que supera a los motores tradicionales como Tesseract en entornos naturales. Su arquitectura se compone de dos sub-redes especializadas:

- **Detección de Texto (CRAFT):** Utiliza el algoritmo "Character Región Awareness for Text Detection". Esta red neuronal estima dos mapas de puntuación: uno para la probabilidad de que un píxel sea el centro de un carácter y otro para la probabilidad de que sea el espacio entre caracteres. Esto permite detectar texto de forma robusta incluso si está curvado o deformado.
- **Reconocimiento de Texto (CRNN):** Una vez localizada la región de texto, se recorta y se pasa a una Red Neuronal Convolutiva Recurrente (CRNN). La parte convolutiva (generalmente una ResNet) extrae características visuales abstractas. Estas características se alimentan a una red recurrente (LSTM - Long Short-Term Memory) que procesa la secuencia de características para predecir la secuencia de caracteres, manejando el contexto secuencial de las letras. Finalmente, se utiliza una capa de CTC (Connectionist Temporal Classification) para decodificar la salida sin necesidad de una segmentación explícita de cada carácter.

(Referencia: EasyOCR Documentation, 2023)

5.3. Gestión de Datos Relacional

La persistencia de los datos se basa en el modelo relacional. Se aplica la teoría de normalización para evitar redundancias. Se garantiza que cada tabla tenga una clave primaria única y que las relaciones entre vehículos y propietarios se manejan mediante claves foráneas (Foreign Keys), asegurando la integridad referencial: no puede existir un registro de detección de placa si el vehículo no ha sido previamente dado de alta o catalogado como "visitante".

(Referencia: Date, 2004)

6. Modelo lógico del sistema

El sistema opera bajo un flujo secuencial de transformación de datos. Matemáticamente, podemos modelar el proceso completo como una función compuesta $R = f(I)$, donde I es la imagen de entrada y R es el resultado estructurado.

El flujo detallado se expresa como:

$$Resultado = Vinculacion(Validacion(OCR(Deteccion(Preprocesamiento(I))))))$$

Desglosando cada etapa del modelo:

1. **Preprocesamiento (P):** $I' = CLAHE(Gris(I))$. Se busca maximizar la varianza local en las regiones de alta frecuencia (bordes de letras).
2. **Detección y OCR (D):** Se obtiene un conjunto de tuplas candidatas $C = \{(texto_1, conf_1, box_1), \dots, (texto_n, conf_n, box_n)\}$.
3. **Validación (V):** Es la etapa de filtrado lógico. Para cada candidato c_i en C , se calcula un **Score** de viabilidad:

$$Score_i = w_1 \cdot Confianza_i + w_2 \cdot PatronMatch_i + w_3 \cdot AspectRatio_i$$

- w_1, w_2, w_3 son pesos heurísticos determinados experimentalmente.
- *PatronMatch* es una función binaria (0 o 1) que evalúa si el texto cumple con los Regex definidos.
- *AspectRatio* evalúa si la caja delimitadora (*box*) tiene proporciones rectangulares típicas de una placa (aprox. 2:1 a 4:1).

Reglas de Negocio Implementadas:

- **Patrones Regex:** Se definen expresiones regulares estrictas para cubrir los formatos vigentes en México.
 - Estándar actual: `^[A-Z]{3}-?\d{3}$` (Tres letras, guion opcional, tres dígitos).
 - Formato anterior/camionetas: `^[A-Z]{2}-?\d{4}$`.
- **Listas de Exclusión (Blacklist):** Se eliminan palabras comunes que aparecen en los vehículos pero no son identificadores únicos, tales como: "SINALOA", "MEXICO", "TRANSPORTE", "PRIVADO", "AUTO", "CAMION".
- **Umbral de Decisión:** Se descartan lecturas con una confianza (probabilidad softmax de la red CRNN) inferior al 40% (0.4). (Referencia: Szeliski, 2010)

7. Propuesta de solución

Para resolver la problemática planteada, se propone una arquitectura de software modular basada en el patrón cliente-servidor, desacoplando la lógica de inferencia pesada (Inteligencia Artificial) de la lógica de gestión de usuarios y presentación.

7.1 Componentes de la Arquitectura

- **Módulo de Inteligencia Artificial (Python Worker):** Consiste en una serie de scripts autónomos (`detect_enhanced.py`, `multi_plate_detector.py`) que actúan

como microservicios. Estos scripts reciben la ruta de una imagen, cargan los modelos neuronales en memoria (o en GPU si está disponible) y emiten la salida en formato JSON estándar a través de la salida estándar (stdout) del sistema. Este diseño permite que el módulo de IA sea agnóstico a la tecnología del servidor web.

- **Backend y Orquestador (Node.js):** Se utiliza un servidor desarrollado en Node.js (archivo `server.js`). Su función principal es exponer endpoints API para la carga de imágenes (`POST /analyze`). Al recibir una petición, el servidor gestiona la carga del archivo, invoca el subproceso de Python (utilizando `child_process.spawn`), captura el resultado JSON y realiza las consultas pertinentes a la base de datos SQL Server.
- **Capa de Datos (SQL Server):** Un motor de base de datos robusto que almacena tres tipos de entidades principales: Propietarios (datos sensibles), Vehículos (datos técnicos y matrícula) y Detecciones (bitácora de eventos con marcas de tiempo).

7.2 Flujo de Información

1. El usuario sube una imagen a través de la interfaz web.
2. Node.js guarda la imagen temporalmente y ejecuta el script de Python.
3. Python procesa la imagen, detecta 5 posibles textos, filtra 4 por ser ruido y retorna 1 matrícula válida con alta confianza.
4. Node.js recibe la matrícula (ej. "VSZ-984"), consulta en SQL Server: `SELECT * FROM Propietarios WHERE Placa = 'VSZ-984'`.
5. Si existe, retorna los datos del dueño; si no, indica "Vehículo no registrado".
(Referencia: Rosebrock, 2018)

8. Desarrollo e implementación

La fase de desarrollo implicó la integración de múltiples tecnologías y la escritura de código específico para manejar las particularidades de las matrículas mexicanas. A continuación, se detallan los módulos críticos desarrollados.

8.1 Preprocesamiento Avanzado (`preprocess.py`)

Se implementó una clase `LicensePlatePreprocessor` que encapsula la lógica de mejora de imagen.

- **Normalización:** Las imágenes entrantes, que pueden variar desde resolución VGA hasta 4K, se redimensionan manteniendo su relación de aspecto para estandarizar el tiempo de inferencia. Se convierten a escala de grises.
- **Mejora de Contraste Local:** Se aplica `cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))`. El `clipLimit` de 2.0 fue seleccionado tras pruebas experimentales; valores más altos introducían demasiado ruido en placas metálicas reflectantes, mientras que valores más bajos no corregía suficientemente las sombras.
- **Reducción de Ruido Selectiva:** Se utiliza `cv2.bilateralFilter(img, 11, 17, 17)`. Los parámetros indican el diámetro del vecindario de píxeles y la desviación estándar en el espacio de color y coordenadas. Esto permite eliminar el

"grano" de las fotos con poca luz sin difuminar los bordes negros de las letras sobre el fondo blanco.

8.2 Detección y OCR Multietapa (detect_enhanced.py)

La detección no se realiza en un solo paso. Se diseñó una estrategia en cascada para maximizar la tasa de éxito:

- **Estrategia 1: Detección de Región (Contornos).** Se intenta localizar rectángulos en la imagen mediante detección de bordes Canny y búsqueda de contornos. Si se encuentra un rectángulo con la relación de aspecto correcta, se recorta y se envía al OCR. Esto es rápido y elimina el ruido del fondo.
- **Estrategia 2: Escaneo Completo.** Si la estrategia 1 falla (por ejemplo, si el parachoques del auto tiene muchos rectángulos), se pasa la imagen completa a EasyOCR. Aunque es más lento, permite que la red neuronal CRAFT encuentre el texto basado en mapas de calor.
- **Estrategia 3: Rotación Automática.** Una innovación clave fue la implementación de un bucle de corrección de rotación. Si las estrategias anteriores fallan, el sistema rota la imagen digitalmente a -10° , -5° , $+5^\circ$ y $+10^\circ$. Esto corrige los casos donde el usuario tomó la foto con el celular inclinado, permitiendo que el OCR lea el texto horizontalmente.
- **Diagnóstico de Errores:** El script incluye un sistema de logging que clasifica los fallos: "Low Confidence", "No Text Found", "Image Too Dark". Esto es vital para la retroalimentación al usuario.

8.3 Validación y Lógica de Negocio (multi_plate_detector.py)

- Se eliminan todos los caracteres no alfanuméricos (espacios, puntos, guiones).
- Se compara la longitud de la cadena resultante. Si es menor a 5 o mayor a 8, se descarta.
- Se aplica el algoritmo de Scoring mencionado en el modelo lógico. Se bonifica (+50 puntos) si coincide con un Regex de placa, se penaliza (-100 puntos) si contiene palabras prohibidas como "FORD" o "NISSAN".
(Referencias: OpenCV Team, 2023; EasyOCR Documentation, 2023)

9. Datos y cartografía de apoyo (Diseño de Base de Datos)

La infraestructura de datos se diseñó utilizando el estándar SQL para asegurar la robustez transaccional. El esquema relacional normalizado en Tercera Forma Normal (3NF) consta de las siguientes tablas clave:

1. Tabla owners (Propietarios):

- id (INT, PK, Identity): Identificador único.
- name (VARCHAR): Nombre completo del propietario.
- contact_info (VARCHAR): Teléfono o email para notificaciones.
- created_at (DATETIME): Fecha de registro.

2. Tabla **vehicles** (Vehículos):

- **id** (INT, PK): Identificador del vehículo.
- **plate_number** (VARCHAR(20), UNIQUE): La matrícula normalizada (sin guiones). Índice agrupado para búsquedas ultra rápidas.
- **owner_id** (INT, FK): Referencia a la tabla **owners**. Relación 1:N (un dueño puede tener varios autos).
- **model** (VARCHAR): Descripción del vehículo (Marca, Modelo, Color).

3. Tabla **detections** (Detecciones):

- **id** (INT, PK): ID del evento.
- **vehicle_id** (INT, FK): Vínculo al vehículo detectado (puede ser NULL si es un desconocido).
- **detected_plate** (VARCHAR): El texto crudo leído por el OCR (útil para auditoría).
- **confidence** (FLOAT): El nivel de confianza reportado por la IA.
- **timestamp** (DATETIME): Momento exacto de la detección.
- **image_path** (VARCHAR): Ruta al archivo de imagen de evidencia.

Este diseño permite realizar consultas complejas, como identificar "vehículos frecuentes no registrados" o "historial de accesos de un propietario específico" mediante JOINS eficientes.

(Referencia: Date, 2004)

10. Métricas y protocolo experimental

Para evaluar objetivamente el desempeño del sistema, se estableció un marco de pruebas riguroso. No basta con que el sistema funcione "a veces"; debe ser consistente.

10.1 Definición de Métricas

- **Precisión (Accuracy):** Se define como la proporción de placas leídas correctamente (coincidencia perfecta de caracteres) sobre el total de placas visibles en el dataset.

$$Accuracy = \frac{TP}{Total\ Muestras}$$

Donde TP (True Positive) es una lectura que coincide exactamente con la verdad terreno (Ground Truth).

- **Robustez Geométrica:** Capacidad del sistema para mantener la precisión cuando la placa presenta rotaciones. Se mide en rangos: $[0^\circ, 5^\circ]$, $[5^\circ, 15^\circ]$ y $> 15^\circ$.
- **Tiempo de Respuesta (Latencia):** El tiempo transcurrido desde que el servidor recibe la petición HTTP hasta que retorna la respuesta JSON. Se busca un objetivo de $t < 3.0$ segundos en CPU estándar.

en cuatro grupos de desafío:

1. **Condiciones Ideales (Grupo A):** Vehículos detenidos, iluminación diurna uniforme, vista frontal directa. (Control).
2. **Iluminación Variable (Grupo B):** Imágenes capturadas al atardecer, en garajes subterráneos o con sombras fuertes proyectadas por árboles.
3. **Perspectiva Forzada (Grupo C):** Imágenes tomadas desde ángulos laterales superiores a 30 grados o desde alturas elevadas (cámaras de seguridad tipo CCTV).
4. **Oclusión y Suciedad (Grupo D):** Placas con barro leve, marcos de concesionaria que cubren parcialmente los bordes, o tornillos oxidados que interfieren con los caracteres.

Durante las pruebas, el script `detect_enhanced.py` se ejecutó con una bandera `--debug`, lo que generó logs detallados de los pasos de binarización y confianza, permitiendo ajustar los umbrales de aceptación del algoritmo.

(Referencia: Szeliski, 2010)

11. Resultados esperados

Basado en el diseño y las pruebas preliminares, se proyectan los siguientes resultados operativos:

1. **Alto Rendimiento en Condiciones Controladas:** Se espera una tasa de reconocimiento superior al **95%** para el Grupo A (Ideal). La combinación de EasyOCR con imágenes limpias es extremadamente fiable.
 2. **Resiliencia ante Iluminación:** Gracias al preprocesamiento CLAHE, se espera mantener una precisión superior al **85%** en el Grupo B (Iluminación Variable), recuperando información legible incluso en zonas de sombra donde un OCR estándar fallaría.
 3. **Recuperación de Perspectiva:** La estrategia de rotación automática debería corregir eficazmente las inclinaciones leves, logrando una precisión aceptable (~80%) en el Grupo C, siempre que la distorsión no sea severa (skew horizontal extremo).
 4. **Eficiencia Computacional:** El tiempo de procesamiento promedio se estima en **1.8 a 2.5 segundos** por imagen en un procesador Intel Core i5 o equivalente sin aceleración GPU, lo cual es aceptable para aplicaciones de control de acceso en barreras de estacionamiento.
 5. **Integridad de Datos:** El sistema garantizará que, ante una placa ilegible o ambigua (confianza < 40%), el sistema prefiera retornar un error explícito ("Placa no detectada") en lugar de un falso positivo, priorizando la seguridad.
- (Referencia: Rosebrock, 2018)

12. Discusión

El análisis de la implementación revela puntos clave sobre la tecnología actual de reconocimiento. El uso de **EasyOCR** demostró ser una decisión acertada frente a alternativas más antiguas como Tesseract. Tesseract requiere una segmentación de

caracteres perfecta y binarización estricta, lo cual falla en "el mundo real" con placas sucias o sombras. EasyOCR, al basarse en mapas de calor de características (CRAFT), es mucho más tolerante al ruido y a la falta de alineación.

Sin embargo, este rendimiento superior tiene un costo: el consumo de recursos. Esto plantea un desafío para implementaciones en tiempo real de alta velocidad (autopistas), sugiriendo que para esos escenarios sería necesario hardware dedicado (GPUs NVIDIA con CUDA).

Otro hallazgo importante fue la importancia crítica de la Lógica de Negocio. El OCR por sí solo es "ingenuo"; leerá cualquier texto que vea (letreros de calles, marcas de autos, calcomanías). Fue la implementación de los filtros Regex y las listas negras lo que transformó un lector de texto genérico en un verdadero Identificador de Matrículas. La rotación automática probó ser una solución algorítmica sencilla pero altamente efectiva para resolver problemas físicos de instalación de cámaras.

(Referencias: EasyOCR Documentation, 2023; OpenCV Team, 2023)

13. Cronograma

El desarrollo del proyecto se estructuró en un plan de trabajo de 5 semanas, asegurando tiempo suficiente para la iteración y corrección de errores.

Semana	Actividad Principal	Detalles de la Actividad
1	Infraestructura de Datos	Instalación de SQL Server. Diseño del diagrama Entidad-Relación. Creación de tablas, índices y procedimientos almacenados iniciales. Carga de datos de prueba (vehículos ficticios).
2	Visión Artificial Básica	Configuración del entorno Python (Anaconda/Virtualenv). Instalación de dependencias (PyTorch, OpenCV). Desarrollo de <code>preprocess.py</code> y pruebas unitarias de filtros CLAHE y Bilateral.
3	Lógica de Detección	Implementación de <code>detect_enhanced.py</code> . Integración de EasyOCR. Programación de las estrategias de cascada (región vs. completo) y corrección de rotación. Ajuste de parámetros.
4	Integración Backend	Desarrollo del servidor API en Node.js. Implementación de <code>child_process</code> para comunicar Node y Python. Conexión a SQL Server mediante driver <code>mssql</code> . Creación de endpoints REST.
5	Pruebas y Documentación	Ejecución del protocolo experimental (dataset de 100 imágenes). Recolección de métricas. Optimización final de código. Redacción del manual técnico y reporte final.

14. Conclusiones

El proyecto "Sistema de Detección de Matrículas" ha logrado cumplir satisfactoriamente con el objetivo de integrar técnicas avanzadas de Inteligencia Artificial con sistemas de gestión de información tradicionales. La arquitectura modular adoptada, que separa claramente la responsabilidad del procesamiento visual (Python) de la gestión de negocio (Node.js/SQL), ha demostrado ser robusta y fácilmente escalable.

Desde una perspectiva técnica, se concluye que en aplicaciones de visión artificial "en la naturaleza" (in the wild), el preprocesamiento de la imagen es tan crítico, o incluso más, que la elección del modelo neuronal. Algoritmos como CLAHE y el filtrado bilateral fueron determinantes para lograr que las imágenes fueran legibles para la red neuronal en condiciones de iluminación adversa. Asimismo, la validación lógica post-OCR se identificó como el componente clave para reducir los falsos positivos, elevando la utilidad práctica del sistema. Este proyecto sienta las bases para futuras mejoras, como la implementación de detección de vídeo en streaming o la migración a dispositivos de borde (Edge AI) para reducir la latencia.

15. Referencias

1. Date, C. J. (2004). *Introduction to Database Systems*. Pearson.
2. EasyOCR Documentation. (2023). Jaided AI. Recuperado de GitHub.
3. OpenCV Team. (2023). *OpenCV Library Documentation*.
4. Rosebrock, A. (2018). *Computer Vision with Python*. PyImageSearch.
5. Szeliski, R. (2010). *Computer Vision: Algorithms and Applications*. Springer.

youtube LINK:

Evidencia1 DeteccionDePlacas

<https://youtu.be/kQpPH44FHCc>

Evidencia2 BaseDeDatos

<https://www.youtube.com/watch?v=nRE2JGdtNDg>